



**KAZAKH-BRITISH
TECHNICAL
UNIVERSITY**

JSC «Kazakh British Technical University»

School of IT and Engineering



SYLLABUS

Discipline: Backend for Highloaded Environment

Term: Fall 2026

Instructor's full name: Nurov Arnur

Personal Information about the Instructor	Time and place of classes		Contact information	
	Lessons	Office Hours	Tel.	e-mail
Nurov Arnur, Lecturer	According to the schedule	According to the schedule		a.nurov@kbtu.kz

COURSE DURATION: 3 credits, 15 weeks, 45 class hours

GENERAL COURSE AIMS

To develop the engineering judgement required to design, build and operate server-side systems that run under heavy load. The course teaches students to make architectural decisions deliberately — choosing a protocol, a storage model and a scaling strategy from the requirements for latency, throughput, consistency and cost — and to defend those decisions with evidence.

COURSE DESCRIPTION

This course covers System Design and the internals of highload backend systems. All practical work is done in Java (Java 21, Spring Boot) with PostgreSQL, Redis, Apache Kafka, Elasticsearch and Docker. The emphasis, however, is not on any single framework but on principles: how network protocols work (TCP/TLS, HTTP/2, HTTP/3, gRPC), how indexes, transactions, MVCC, replication and sharding behave, how caching, message queues, distributed algorithms and resilience mechanisms are built, and how all of this is measured and observed in production.

Each week combines two hours of lectures with one hour of practice, where theory is immediately tested by measurement: load tests, profiling and query plan analysis. Independent study (SIS) is a chain of four engineering assignments — designing a system, implementing it with an appropriate protocol, tuning its database, and scaling it out on Kubernetes — while the final exam is oral and ticket-based.

COURSE PRE REQUISITES

Confident Java programming (OOP and collections); basic SQL and relational database knowledge; familiarity with computer networking fundamentals; experience with the command line and Git. The course is an elective open to a mixed audience, so the first two weeks include levelling material on measurement tooling and the network stack.

COURSE POST REQUISITES: -

COURSE OBJECTIVES

- To teach students to state non-functional requirements (SLA/SLO/SLI, target p95/p99, RPS, data volume) and to produce capacity estimates.
- To build an understanding of transport and application layer protocols and of the criteria for choosing between REST, gRPC and streaming protocols.
- To develop a deep understanding of database behaviour: indexes, transactions, isolation levels, MVCC, replication, partitioning and sharding.
- To teach students to apply caching, message queues and asynchronous processing to reduce load and increase resilience.
- To introduce the principles of distributed systems: consistency, consensus, idempotency, sagas and fault tolerance.
- To build practical skills in measurement and observability: load testing, metrics, tracing, JVM profiling and GC tuning.
- To develop the ability to defend architectural decisions orally, as in an engineering review or a System Design interview.

COURSE OUTCOMES

Upon completion of the course a student is able to:

- CLO1 — design the architecture of a service for a given load and response-time requirement and justify each decision quantitatively;
- CLO2 — select and correctly apply a network protocol and serialization format, explaining the trade-offs in latency, throughput and compatibility;
- CLO3 — design a data schema, indexes and a database scaling strategy, and diagnose slow queries and consistency problems;
- CLO4 — apply caching, queues and resilience patterns (rate limiting, circuit breaker, bulkhead, retry with backoff) and evaluate their effect through measurement;
- CLO5 — instrument a service for observability, run a load test, locate the bottleneck and remove it;
- CLO6 — defend an architectural decision orally under clarifying and challenging questions.

LITERATURE

Core:

- Kleppmann M. Designing Data-Intensive Applications. O'Reilly, 2017.
- Xu A. System Design Interview – An Insider's Guide, Vol. 1–2. ByteByteGo, 2020–2022.
- Petrov A. Database Internals: A Deep Dive into How Distributed Data Systems Work. O'Reilly, 2019.
- Goetz B. et al. Java Concurrency in Practice. Addison-Wesley, 2006.

Supplementary:

- Nygard M. Release It! Design and Deploy Production-Ready Software. 2nd ed., Pragmatic Bookshelf, 2018.
- Beyer B. et al. Site Reliability Engineering: How Google Runs Production Systems. O'Reilly, 2016.

- Narkhede N., Shapira G., Palino T. Kafka: The Definitive Guide. 2nd ed., O'Reilly, 2021.
- Grigorik I. High Performance Browser Networking. O'Reilly, 2013 (chapters on TCP, TLS and HTTP/2).
- Official documentation: PostgreSQL, Redis, Apache Kafka, Spring Boot, OpenTelemetry, Kubernetes.

COURSE ASSESSMENT CRITERIA

Assessment occurs continuously throughout the course. The evaluation will be based on the levels of (maximums in %):

Type of activity	Final scores
Attendance / participation	10 %
SIS (design & implementation assignments)	20 %
Midterm / End-term (tests)	25 %
Quiz	5 %
Final exam (ticket-based, oral)	40 %
Total	100 %

TASKS for students' independent study (SIS)

SIS is a chain of four engineering assignments built on top of one another: the student designs a system, implements it with an appropriate protocol, tunes its database and finally scales it out and makes it resilient. Each assignment is submitted as a repository plus a short report with measurements, and is briefly discussed with the instructor. Work that cannot be explained by its author is not accepted.

Week	SIS	Cost (in points)
6	SIS #1 — System design. For a given set of non-functional requirements (target RPS, data volume, latency budget) produce a design document: capacity estimates, API contract, chosen scaling type, data model, caching strategy, failure modes and architecture diagrams	5
8	SIS #2 — Protocol implementation. Implement the service interface in Java using the protocol justified in SIS #1 (gRPC / HTTP-2 / WebSocket), with idempotency, timeouts and retries; submit measurements of latency and payload size against a REST/JSON baseline	5
13	SIS #3 — Database tuning. Design the schema and indexes for the workload, analyse query plans with EXPLAIN ANALYZE, apply partitioning, read replicas and connection pooling; submit a before/after benchmark with p95/p99 and an explanation of every gain	5
14	SIS #4 — Scale-out and resilience. Extend the service with caching, asynchronous processing (Kafka) and resilience patterns, deploy it to Kubernetes with horizontal pod autoscaling, run a load test and submit a bottleneck analysis	5
	Total	20

Grading criteria for a single SIS assignment (5 points):

- 5 — the solution meets the stated requirements, every decision is justified quantitatively, measurements are reproducible and the author explains the trade-offs fluently;
- 4 — a working solution with minor design or implementation flaws, or with weak quantitative reasoning;
- 3 — the solution works only under simplified assumptions; measurements are incomplete or superficial;

- 1-2 — the requirements are largely unmet, or the report contains no evidence of measurement;
- 0 — the work was not submitted, or the student cannot explain their own solution.

TASKS for practice

Week	Practice task
1	Baseline Java service (Spring Boot): measuring p50/p95/p99; capacity estimation for a target RPS; comparing one large instance against several small ones behind a balancer
2	Load testing HTTP/1.1 vs HTTP/2 vs HTTP/3 (k6/wrk): connection pooling and the cost of the TLS handshake
3	Building a gRPC service in Java with Protobuf; comparing it to REST/JSON by latency and payload size
4	JMH benchmark: thread pool vs virtual threads under load; diagnosing contention. Quiz 1
5	Redis and Caffeine: cache-aside, stampede protection, measuring hit ratio and its effect on p99
6	PostgreSQL: EXPLAIN ANALYZE, index design, eliminating N+1 and slow queries
7	Read replicas with PgBouncer/HikariCP, table partitioning, observing the effects of replication lag
8	Data modelling for Cassandra/MongoDB against a given access pattern. Midterm
9	Kafka in Java: producer/consumer, at-least-once delivery with an idempotent consumer, the outbox pattern
10	Distributed lock on Redis/ZooKeeper, saga orchestration, idempotency keys
11	Resilience4j: rate limiter, circuit breaker, bulkhead; configuring Nginx/Envoy as an L7 balancer
12	OpenTelemetry with Prometheus and Grafana; JFR profiling and GC tuning. Quiz 2
13	Deploying the Java service to Kubernetes (kind/minikube): manifests, ConfigMaps and secrets, service and ingress, rolling update
14	HPA under a load test: tuning probes and limits, observing scale-out behaviour, performing a canary rollout
15	Elasticsearch and S3-compatible object storage; whiteboard design workshop and a mock ticket-based exam. End-term
	Total

COURSE CALENDAR

Week	Topic	Lectures (hours)	Practice (hours)	SIS defense
1	Introduction to highload and types of scaling. Vertical vs horizontal scaling and the limits of scaling up; stateless vs stateful services; scaling reads vs scaling writes; functional decomposition vs data sharding; SLA/SLO/SLI, latency vs throughput, tail latency (p95/p99), back-of-the-envelope estimation, Amdahl's Law and the Universal Scalability Law	2	1	
2	Networking and protocols I. TCP/IP, handshake and congestion control, TLS 1.3, HTTP/1.1 keep-alive, HTTP/2 multiplexing, HTTP/3 and QUIC, WebSocket, SSE	2	1	
3	Networking and protocols II. REST vs gRPC vs GraphQL, Protobuf and serialization, schema evolution and API versioning, request idempotency, timeouts and retries	2	1	
4	Concurrency on the JVM. Memory model, thread pools, virtual threads (Project Loom), blocking vs reactive, backpressure, contention and false sharing	2	1	
5	Caching. Cache-aside / write-through / write-behind, Redis, invalidation strategies and TTL, cache stampede, multi-level caches, CDN	2	1	
6	Databases I. B-trees and indexes, the query planner, transactions, isolation levels, MVCC in PostgreSQL, locking and deadlocks	2	1	SIS #1 due
7	Databases II. Replication (sync/async), read replicas and replication lag, failover, sharding and partitioning, connection pooling	2	1	
8	NoSQL and the theory of distributed data. CAP and PACELC, key-value / document / wide-column stores, consistency models, quorum reads and writes	2	1	SIS #2 due
9	Asynchronous architecture. Queues and brokers, Apache Kafka: topics, partitions, consumer groups, delivery semantics, transactional outbox, CDC	2	1	
10	Foundations of distributed systems. Time and clocks, consensus (Raft), leader election, distributed locks, 2PC vs Saga, why exactly-once is an illusion	2	1	
11	Scaling, resilience and the edge. L4/L7 load balancing, service discovery, autoscaling, rate limiting, circuit breaker, bulkhead, graceful degradation, DDoS protection, AuthN/AuthZ at scale	2	1	
12	Observability and JVM performance. Metrics, logs, distributed tracing (OpenTelemetry), profiling, GC tuning, SRE practices	2	1	

13	Kubernetes and container orchestration I. Why orchestration is required at scale; cluster architecture: control plane, etcd, scheduler, kubelet; pods, deployments and ReplicaSets; services, ingress and in-cluster service discovery; declarative model and the reconciliation loop	2	1	SIS #3 due
14	Kubernetes for highload II. Resource requests and limits, QoS classes and the effect of CPU throttling on p99; liveness/readiness/startup probes and graceful shutdown; horizontal pod autoscaling and cluster autoscaling; rolling, blue-green and canary releases; running stateful workloads and databases in a cluster; cost of capacity	2	1	SIS #4 due
15	Data storage and search; course synthesis. LSM-tree vs B-tree, full-text search (Elasticsearch), object storage, time-series and analytical stores; System Design methodology and end-to-end case review (news feed, URL shortener, ride-hailing, chat)	2	1	
Total		30	15	
16-17	Final Exam			

Course assessment schedule

No.	Assessment criteria	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	Total
1	Attendance / participation	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	10 %
2	SIS (design & implementation)					*		*					*	*			20 %
3	Midterm / End-term (tests)							*								*	25 %
4	Quiz				*								*				5 %
5	Final exam (ticket-based, oral)																40 %
	Total																100 %

Class sessions — a mixture of instruction, analysis of real production incidents and architectural case studies, and discussion of the trade-offs between competing solutions.

Practice — weekly hands-on work in Java: implement, measure, analyse. A task is accepted only when a working result is delivered together with measurements, not code alone. Work is submitted by the end of the week; late submissions receive 50 % of the task value.

In-class assessment — short questioning and walkthroughs that prepare students for the attestations and expose gaps in understanding.

SIS (Student Independent Study) — four cumulative engineering assignments submitted in weeks 6, 8, 13 and 14: system design, protocol implementation, database tuning, and scale-out with resilience. Each is delivered as a repository and a short measurement report. Guidance is available during office hours.

Quiz — computer-based tests in weeks 4 and 12 covering the material studied to that point.

Midterm / End-term — attestations in weeks 8 and 15: a written paper with design problems and estimation tasks.

Final examination — an oral ticket-based exam. The student draws a ticket, prepares for 20 minutes and answers for 15–20 minutes before the examination board. Each ticket contains two theoretical questions drawn from the whole course (scaling types, protocols, databases, distributed systems, Kubernetes, observability) and one System Design case to be sketched on the board and defended under follow-up questions. Maximum 40 points: 10 points per theoretical question and 20 points for the case. The ticket bank is published in MS Teams no later than week 12.

Grading of the System Design case: correctness and completeness of the design against the stated requirements (8 points), quantitative justification of capacity, protocol and storage decisions (6 points), treatment of failure modes and scaling strategy (4 points), and quality of answers to follow-up questions (2 points).

Grading policy

Intermediate attestations (on 8th and 15th week) join topics of all lectures, practice, SIS and materials for reading discussed to the time of attestation. Maximum number of points within attendance, participation, practice, SIS and quiz for each attestation is 30 points.

Final exam joins and generalizes all course materials and is conducted orally: the student draws a ticket, prepares for 20 minutes and answers for 15–20 minutes. Maximum number of points is 40. At the end of the semester you receive overall total grade (summarized index of your work during semester) according to conventional KBTU grade scale.

ATTENTION!

- Attendance is mandatory for every class of the course. Presence is recorded in each session; an absence without a valid reason forfeits the attendance points and the practice points for that week.
- If student missed more than 30% of lessons student receives «F (Fail)» grade;
- If for two attestations student receives 29.4 or less points, this student is not accepted to final exam and for all course he (she) receives «F (Fail)» grade;
- If student receives 9.4 or less points on final exam, then independently on how many points he (she) received for two attestations, in whole he (she) receives «F (Fail)» grade; In the case of missing or being late for final exam without plausible reason, independently on how many points he (she) received for two attestations, in whole he (she) receives «F (Fail)» grade.

ACADEMIC POLICY

- Cheating, duplication, falsification of data, plagiarism are not permitted under any circumstances!
- Attendance is mandatory for all classes. Students must participate fully in every class. While attendance is crucial, merely being in class does not constitute “participation”. Participation means reading the assigned materials, coming to class prepared to ask questions and engage in discussion.
- Students are expected to take an active role in learning (the instructor will provide the information and guidelines to do this).
- Students must come to class on time.
- Students are to take responsibility for making up any work missed.
- Make up tests in case of absence will not normally be allowed.
- Mobile phones must always be switched off in class.
- Students should always show tolerance, consideration and mutual support towards other students.

Generative AI Use — Level D (AI-Integrated) for practical work and code.

Generative AI is permitted and encouraged as a working tool at every stage of the practical assignments: generating code and configuration, writing tests and load scenarios, debugging,

explaining concepts, and searching and analysing material. This course is about how architectural decisions are made, not about how fast boilerplate is typed, so accelerating development with AI is part of the expected workflow.

Conditions of use:

- The student carries full responsibility for the submitted work and must review, adapt and understand every line of the result.
- The README of each practical assignment states briefly (2–3 sentences) where and how AI was used.
- Oral SIS ticket defenses, quizzes, attestations and the final exam are conducted without AI or any other assistance — these are what verify genuine understanding.
- If a student cannot explain the logic, the trade-offs or the behaviour of their own solution when questioned orally, the corresponding work receives 0 points regardless of its quality.
- Reliance on AI is not an excuse for factual errors, fabricated sources or code that does not work.

Grade		Achievement percentage	Assessment criterion
«Excellent»	A	95-100%	This grade is given when the student: - demonstrated a complete understanding of the course material; - did not make any errors or inaccuracies; - completed control and laboratory work in a timely and correct manner, and submitted reports on them; - demonstrated original thinking; - submitted control quizzes on time and without any errors; - completed homework assignments; - engaged in research work; - independently used additional scientific literature in studying the discipline; - was able to independently systematize the course material.
	A -	90-94%	
«Good»	B+	85-89%	This grade is given when the student: - Has mastered the course material at no less than 75%; - Did not make gross errors in responses; - Timely completed control and laboratory work and submitted them without fundamental remarks; - Correctly completed and timely submitted control tests and homework assignments without fundamental remarks; - Utilized additional literature as indicated by the instructor; - Engaged in research work, made non-fundamental errors, and fundamental errors corrected by the student themselves; - Managed to systematize the course material with the help of the instructor.
	B	80-84%	
	B-	75-79%	
	C+	70-74%	

«Satisfactory»	C	65-69%	This grade is given when the student: Has mastered the course material no less than 50%; Required assistance from the instructor when completing control and laboratory work, homework assignments; Made inaccuracies and non-fundamental errors when submitting control tests; Did not demonstrate activity in research work, relied solely on the educational literature indicated by the instructor; Experienced more difficulty in systematizing the material.
	C-	60-64%	
	D+	55-59%	
	D	50-54%	

Instructor

Nurov A A

Nurov Arnur

**Professor, Academic Supervisor of the
Computer Science Division, SITE**

Pakizar Shamoi

Pakizar Shamoi

Minutes # 1 of School of Information Technology and Engineering meeting on August 18, 2026